

COMPTE RENDU DE TRAVAUX PRATIQUES DE CYBERSECURITE

Analyse du trafic réseau avec Wireshark

1. Introduction

Wireshark est un logiciel libre et open-source spécialisé dans l'analyse des protocoles réseau. Il permet de capturer et d'examiner en détail les paquets de données qui circulent sur un réseau informatique. Grâce à son interface graphique intuitive et ses puissantes fonctionnalités. Le rôle principal de Wireshark est de fournir une visibilité complète sur les communications réseau. Il décompose chaque paquet pour en révéler les informations essentielles : adresses IP, ports, protocoles utilisés, contenu des échanges, etc. Cela permet de Surveiller le trafic en temps réel, Identifier les anomalies ou erreurs de communication.

L'analyse réseau avec Wireshark est cruciale pour plusieurs raisons : elle aide à localiser rapidement la source d'un problème de performance ou de connectivité, elle permet de détecter des activités suspectes, des intrusions ou des attaques, elle contribue à améliorer la configuration et l'efficacité du réseau en identifiant les goulots d'étranglement, elle assure que les communications respectent les normes et politiques de sécurité.

2. Description des manipulations

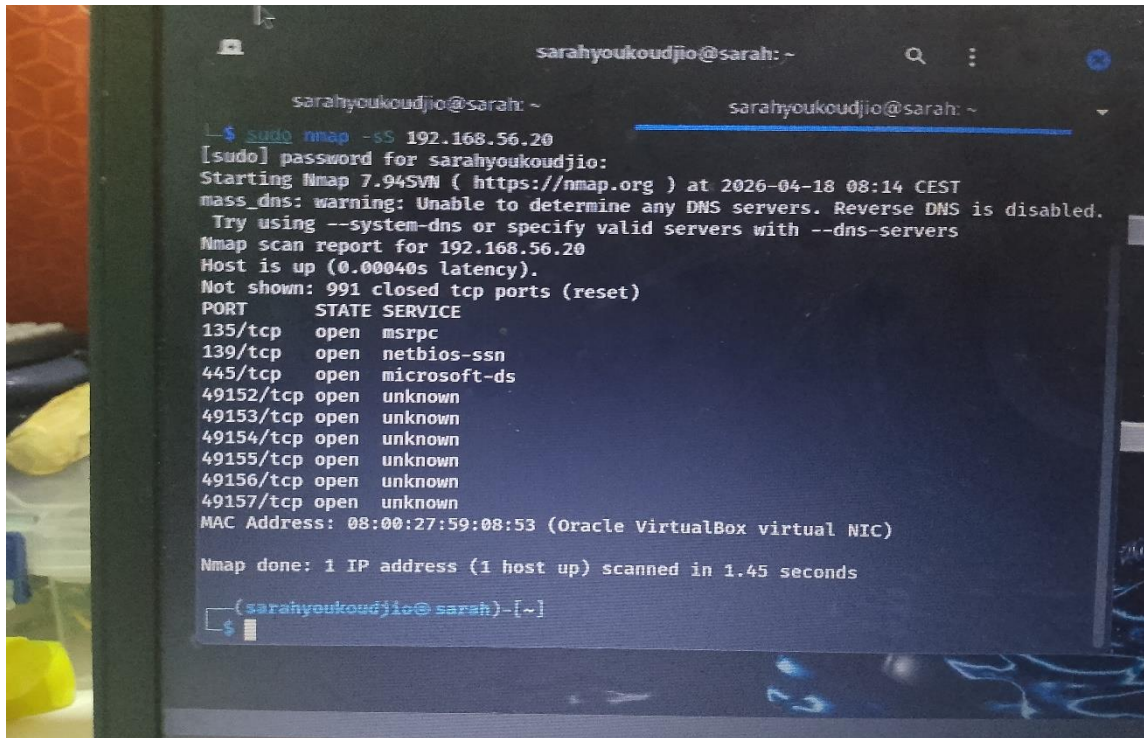
La manipulation a été fait en plusieurs étapes :

- Étapes 1 : lancer wireshark grâce commande wireshark
- Étapes 2 : Double cliqué sur l'interface réseau utiliser par le réseau qui est ici eth0
- Étapes 3 : Taper la commande nmap -sS 192.168.56.20 pour avoir un trafic identifiable à analyser.

3. Résultats obtenus

3.1 Capture du scan Nmap

Le scan Nmap a retourné les résultats suivants :



3.2 Capture Wireshark — Traffic brut

No.	Time	Source	Destination	Protocol	Length	Info
1755	108.868330821	192.168.56.40	192.168.56.30	TCP	60	5802 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1756	108.868331157	192.168.56.40	192.168.56.30	TCP	60	19350 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1757	108.870711381	192.168.56.30	192.168.56.40	TCP	58	56962 → 515 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1758	108.870806955	192.168.56.30	192.168.56.40	TCP	58	56962 → 32774 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1759	108.871524262	192.168.56.30	192.168.56.40	TCP	58	56962 → 2251 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1760	108.871626247	192.168.56.30	192.168.56.40	TCP	58	56962 → 40193 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1761	108.872388035	192.168.56.40	192.168.56.30	TCP	60	515 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1762	108.872388327	192.168.56.40	192.168.56.30	TCP	60	32774 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1763	108.872557074	192.168.56.30	192.168.56.40	TCP	58	56962 → 7938 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1764	108.872660998	192.168.56.30	192.168.56.40	TCP	58	56962 → 668 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1765	108.873653521	192.168.56.30	192.168.56.40	TCP	58	56962 → 65389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1766	108.873763158	192.168.56.30	192.168.56.40	TCP	58	56962 → 1042 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1767	108.875200471	192.168.56.40	192.168.56.30	TCP	60	2251 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1768	108.875200797	192.168.56.40	192.168.56.30	TCP	60	40193 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1769	108.875200907	192.168.56.40	192.168.56.30	TCP	60	7938 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1770	108.875200990	192.168.56.40	192.168.56.30	TCP	60	668 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1771	108.876576641	192.168.56.40	192.168.56.30	TCP	60	65389 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1772	108.876576902	192.168.56.40	192.168.56.30	TCP	60	1042 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1773	108.886929227	192.168.56.30	192.168.56.40	TCP	58	56962 → 5960 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1774	108.887036386	192.168.56.30	192.168.56.40	TCP	58	56962 → 50300 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1775	108.887810749	192.168.56.30	192.168.56.40	TCP	58	56962 → 2525 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1776	108.887940175	192.168.56.30	192.168.56.40	TCP	58	56962 → 22939 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1777	108.888830999	192.168.56.40	192.168.56.30	TCP	60	5960 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1778	108.888831286	192.168.56.40	192.168.56.30	TCP	60	50300 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1779	108.888991903	192.168.56.30	192.168.56.40	TCP	58	56962 → 32775 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1780	108.889091498	192.168.56.30	192.168.56.40	TCP	58	56962 → 5959 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1781	108.889959971	192.168.56.40	192.168.56.30	TCP	60	2525 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Interprétation : On observe un volume important de paquets TCP générés en très peu de temps, caractéristique d'un scan de ports. Les paquets sont envoyés séquentiellement vers chaque port de la cible.

3.3 Capture Wireshark — Filtre SYN

tcp.flags.syn==1						
No.	Time	Source	Destination	Protocol	Length	Info
7	107.659015342	192.168.56.30	192.168.56.40	TCP	58	56962 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
8	107.659142770	192.168.56.30	192.168.56.40	TCP	58	56962 → 110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
9	107.659906439	192.168.56.30	192.168.56.40	TCP	58	56962 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	107.660006489	192.168.56.30	192.168.56.40	TCP	58	56962 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	107.661161793	192.168.56.30	192.168.56.40	TCP	58	56962 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	107.662021195	192.168.56.40	192.168.56.30	TCP	60	80 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
18	107.663976014	192.168.56.30	192.168.56.40	TCP	58	56962 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	107.664079145	192.168.56.30	192.168.56.40	TCP	58	56962 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	107.665245831	192.168.56.30	192.168.56.40	TCP	58	56962 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	107.665344784	192.168.56.30	192.168.56.40	TCP	58	56962 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
26	107.667227475	192.168.56.30	192.168.56.40	TCP	58	56962 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
27	107.669769705	192.168.56.40	192.168.56.30	TCP	60	22 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
29	107.671647773	192.168.56.30	192.168.56.40	TCP	58	56962 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	107.671774634	192.168.56.30	192.168.56.40	TCP	58	56962 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
31	107.672542508	192.168.56.30	192.168.56.40	TCP	58	56962 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	107.672648937	192.168.56.30	192.168.56.40	TCP	58	56962 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	107.678539774	192.168.56.30	192.168.56.40	TCP	58	56962 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	107.678666004	192.168.56.30	192.168.56.40	TCP	58	56962 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	107.679418947	192.168.56.30	192.168.56.40	TCP	58	56962 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
41	107.680537772	192.168.56.30	192.168.56.40	TCP	58	56962 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
42	107.680639513	192.168.56.30	192.168.56.40	TCP	58	56962 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	107.690549104	192.168.56.30	192.168.56.40	TCP	58	56962 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
48	107.690652897	192.168.56.30	192.168.56.40	TCP	58	56962 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
49	107.691330329	192.168.56.30	192.168.56.40	TCP	58	56962 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
50	107.691434348	192.168.56.30	192.168.56.40	TCP	58	56962 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
53	107.693083876	192.168.56.30	192.168.56.40	TCP	58	56962 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
54	107.693193900	192.168.56.30	192.168.56.40	TCP	58	56962 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Interprétation : Après application du filtre SYN, seuls les paquets initiateurs de connexion TCP sont visibles. On constate que Kali envoie des SYN vers chaque port de la cible de manière séquentielle ou parallèle selon la configuration du scan.

3.4 Capture Wireshark SYN-ACK

tcp.flags.syn==1 && tcp.flags.ack==1						
No.	Time	Source	Destination	Protocol	Length	Info
14	107.662021195	192.168.56.40	192.168.56.30	TCP	60	80 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
27	107.669769705	192.168.56.40	192.168.56.30	TCP	60	22 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460

Interprétation : le syn-ack n'est présent que sur deux ports 80 et 22 ce qui correspond au deux ports ouvert donne par nmap ; donc le syn-ack correspond donc à un port ouvert.

3.5 Capture Wireshark RST

tcp.flags.reset						
o.	Time	Source	Destination	Protocol	Length	Info
13	107.662021070	192.168.56.40	192.168.56.30	TCP	60	110 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
14	107.662021195	192.168.56.40	192.168.56.30	TCP	60	80 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=6424
15	107.662021299	192.168.56.40	192.168.56.30	TCP	60	113 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
16	107.662050347	192.168.56.30	192.168.56.40	TCP	54	56962 → 80 [RST] Seq=1 Win=0 Len=0
17	107.663427385	192.168.56.40	192.168.56.30	TCP	60	199 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
18	107.663976014	192.168.56.30	192.168.56.40	TCP	58	56962 → 1723 [SYN] Seq=0 Win=1024 Len=0 MS
19	107.664079145	192.168.56.30	192.168.56.40	TCP	58	56962 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=
20	107.665050416	192.168.56.40	192.168.56.30	TCP	60	1723 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0
21	107.665050906	192.168.56.40	192.168.56.30	TCP	60	554 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
22	107.665245831	192.168.56.30	192.168.56.40	TCP	58	56962 → 3306 [SYN] Seq=0 Win=1024 Len=0 MS
23	107.665344784	192.168.56.30	192.168.56.40	TCP	58	56962 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=
24	107.667010822	192.168.56.40	192.168.56.30	TCP	60	3306 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0
25	107.667011086	192.168.56.40	192.168.56.30	TCP	60	21 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Le
26	107.667227475	192.168.56.30	192.168.56.40	TCP	58	56962 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=
27	107.669769705	192.168.56.40	192.168.56.30	TCP	60	22 → 56962 [SYN, ACK] Seq=0 Ack=1 Win=6424
28	107.669813465	192.168.56.30	192.168.56.40	TCP	54	56962 → 22 [RST] Seq=1 Win=0 Len=0
29	107.671647773	192.168.56.30	192.168.56.40	TCP	58	56962 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS
30	107.671774634	192.168.56.30	192.168.56.40	TCP	58	56962 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS
31	107.672542508	192.168.56.30	192.168.56.40	TCP	58	56962 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS
32	107.672648937	192.168.56.30	192.168.56.40	TCP	58	56962 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=
33	107.674525806	192.168.56.40	192.168.56.30	TCP	60	443 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
34	107.674526171	192.168.56.40	192.168.56.30	TCP	60	445 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
35	107.676703958	192.168.56.40	192.168.56.30	TCP	60	587 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 L
36	107.676704300	192.168.56.40	192.168.56.30	TCP	60	25 → 56962 [RST, ACK] Seq=1 Ack=1 Win=0 Le

Interprétation : Pour un port fermé, la réponse est un RST immédiat

4. Analyse technique

- ❖ **Un paquet SYN (Synchronize)** est le premier message échangé lors de l'établissement d'une connexion TCP. Il active le drapeau SYN dans l'en-tête TCP et transporte un numéro de séquence initial (ISN) choisi par le client. Ce mécanisme permet de synchroniser les numéros de séquence entre les deux hôtes et d'initier la fameuse « poignée de main à trois voies » (SYN → SYN-ACK → ACK). Dans le cadre d'un **scan SYN avec Nmap (-sS)**, ce paquet est utilisé pour tester l'état d'un port sans finaliser la connexion :
 - **SYN-ACK reçu** → le port est considéré comme **ouvert**.
 - **RST reçu** → le port est **fermé**.
 - **Absence de réponse ou blocage** → le port est **filtré** par un pare-feu ou un dispositif de sécurité.
- ❖ Un **port est considéré comme ouvert** lorsqu'un service actif écoute sur ce port et accepte les connexions entrantes. Lorsqu'un client envoie un paquet **SYN**, la cible qui a un service en écoute répond par un **SYN-ACK**, indiquant qu'elle est prête à établir la communication. Cette séquence est visible dans **Wireshark** sous la forme : SYN (source → cible), Suivi de SYN-ACK (cible → source)
- ❖ Un **scan réseau**, et plus particulièrement un **scan SYN**, peut être identifié grâce à plusieurs signes visibles dans une capture Wireshark ou par un système IDS. On observe généralement :
 - **Un envoi rapide et massif de paquets SYN** provenant d'une même adresse IP vers une multitude de ports différents, ce qui traduit une tentative d'exploration.
 - **L'absence de complétion du handshake TCP** : après réception d'un SYN-ACK, aucun ACK n'est envoyé, ce qui est typique du scan SYN dit "semi-ouvert".
 - **De nombreux paquets RST en retour**, générés par les ports fermés qui rejettent la tentative de connexion.
 - **Des intervalles de temps très courts entre les paquets**, produisant des rafales de trafic inhabituelles et facilement détectables.

5. Utilisation avancée de Wireshark

Filtre 1 : `ip.addr == 192.168.56.10`

- **Utilité :** Permet d'isoler tout le trafic entrant et sortant d'une adresse IP spécifique. C'est idéal pour se concentrer sur une machine cible dans un environnement où plusieurs hôtes communiquent simultanément.
- **Exemple concret :** Lors d'une analyse de compromission, ce filtre permet de suivre uniquement les échanges entre l'attaquant identifié (192.168.56.10) et le serveur victime, en excluant les autres flux du réseau.

Filtre 2 : `tcp.port == 80`

- **Utilité :** Sert à analyser le trafic HTTP, puisque le port 80 est le port par défaut du protocole **HTTP (Hypertext Transfer Protocol)**. Ce filtre permet de visualiser les requêtes et réponses web entre client et serveur.
- **Exemple concret :** Lors d'un test d'intrusion, ce filtre met en évidence les requêtes HTTP envoyées par un navigateur vers une application web vulnérable, révélant par exemple des paramètres transmis en clair dans une requête GET.

• **Filtre 3 :** `tcp.flags.reset == 1`

- **Utilité :** Identifie tous les paquets **RST (Reset)** échangés sur le réseau. Un volume élevé de RST est un indicateur typique d'un scan de ports ou de tentatives de connexion vers des services inexistantes.
- **Exemple concret :** Pendant une tentative de connexion à un service non disponible, ce filtre montre les multiples réponses RST envoyées par le serveur, confirmant que les ports ciblés sont fermés mais accessibles.

6. Lien avec la cybersécurité en entreprise

6.1 Comment un SOC utilise Wireshark

Dans un SOC, Wireshark est un outil d'investigation réseau qui complète les systèmes de détection automatisés. Il permet aux analystes de :

- Confirmer **les alertes** : en vérifiant directement les paquets suspects signalés par un IDS ou un SIEM.
- Reconstituer **les incidents** : en retraçant les échanges réseau pour comprendre comment une attaque s'est déroulée et quelles machines ont été impliquées.
- Appuyer **les enquêtes forensiques** : en fournissant des preuves précises et détaillées sur le trafic réseau lors d'un incident de sécurité.

6.2 Comment détecter une attaque réseau

La détection d'une attaque réseau dans un environnement d'entreprise repose sur plusieurs indicateurs à surveiller via Wireshark ou un IDS :

- Scan de ports : burst de SYN vers de nombreux ports depuis une même source
- Exfiltration de données : volumes de trafic sortant inhabituels vers des IPs externes inconnues.

7. Conclusion

Ce TP m'a permis de relier la théorie du protocole TCP à une pratique concrète. En manipulant Nmap et Wireshark, j'ai compris comment un simple scan se traduit par une succession de paquets et comment chaque réponse révèle l'état réel d'un service. L'exercice m'a montré que l'analyse réseau ne se limite pas à lancer une commande, mais qu'elle exige une lecture attentive des échanges pour en tirer des indices fiables. Wireshark, bien qu'extrêmement puissant pour visualiser le trafic, reste un outil d'observation : c'est à l'analyste d'interpréter les données et de donner du sens aux alertes. Cette approche m'a donné une vision plus professionnelle de la reconnaissance réseau, proche de celle d'un SOC ou d'un pentester, où chaque paquet est une pièce du puzzle de la sécurité.